

Adaptive double-phase ROF model: report on optimising the parameters and comparison with other models

Wojciech Górny* Michał Łasica† Alexandros Matsoukas‡

May 8, 2026

Abstract

This note reports the results of several numerical experiments involving the adaptive double-phase ROF model in the two-dimensional case. We discuss the optimisation of parameters within the model and compare the performance of the adaptive double-phase model and its variants with respect to other image denoising models.

1 Introduction

Below, we briefly describe some numerical experiments concerning the adaptive double-phase ROF model of image denoising; for the detailed description of the model and the definition of weights or parameters we refer to the introduction of the report file for 1D experiments. In this report, we describe how the adaptive double-phase ROF model compares to other standard models in image denoising and discuss several possible modifications to the model. The numerical experiments reported here were done using an implementation of the adaptive double-phase ROF model through the accelerated Chambolle–Pock algorithm [5]. The classical Rudin–Osher–Fatemi [7] and Huber-ROF [4] models were also implemented using the Chambolle–Pock algorithm, as well as for the generalised total variation model [2]; the Chen-Levine-Rao model was implemented using the finite differences method; and the non-local means (NL-means) model [3] was implemented using the standard `denoise_nl_means` command in the `scikit-image` Python library.

We present the results visually, as well as compare the values of the measures SSIM [9], LPIPS [10], and PSNR (see e.g. [8]). To compare the performance of the different models, we primarily use the BSDS500 dataset. The algorithm was implemented in python locally and using the Google Colab platform.

*Faculty of Mathematics, Universität Wien, Oskar-Morgerstern-Platz 1, 1090 Vienna, Austria; Faculty of Mathematics, Informatics and Mechanics, University of Warsaw, Banacha 2, 02-097 Warsaw, Poland; wojciech.gorny@univie.ac.at

†Institute of Mathematics of the Polish Academy of Sciences, Śniadeckich 8, 00-656 Warsaw, Poland; młasica@impan.pl

‡Department of Mathematics, School of Applied Mathematical and Physical Sciences, National Technical University of Athens, Zografou Campus, 157 80 Athens, Greece; alexmatsoukas@mail.ntua.gr

2 Optimisation of parameters

In the first part of the report, we describe how to choose the parameters in the adaptive double-phase ROF model. The first stage is the optimisation of the parameter λ in the classical ROF model [7], i.e.,

$$\min_u \int |\nabla u| dx + \frac{1}{2\lambda} \int |u - g|^2 dx.$$

(We denote it by λ_{ROF} whenever there is need for distinguish several different parameters.) The choice of λ depends in a non-explicit way on the noise level. Assuming that the noise is Gaussian with standard deviation σ , λ increases with as σ increases but no exact rate is given. We thus computed the average values of the SSIM, LPIPS, and PSNR metrics on the “train” part of the BSDS500 dataset and picked the optimal values of λ depending on the noise level. The optimisation of SSIM and LPIPS gave results most consistent with the quality of the visual output and we present them below. We used the same technique also for optimisation of the other models, which possibly contain more than one free parameter.

2.1 The classical ROF model

The first reported experiments concern the optimisation of the parameter λ in the classical ROF model. In Tables 1 and 2, we present the results of optimisation of λ with respect to the SSIM and LPIPS metrics respectively.

Noise variance σ^2	Optimal λ	Maximal SSIM value
0.001	0.02	0.9113
0.002	0.03	0.8714
0.004	0.045	0.8200
0.007	0.06	0.7694
0.01	0.075	0.7334
0.02	0.115	0.6573
0.04	0.165	0.5722
0.07	0.215	0.5016
0.1	0.25	0.4556
0.2	0.315	0.3699

Table 1: Results of optimisation of λ using SSIM

Observe that, for small and medium noise levels (the first five entries), approximately the optimal values of λ (using SSIM) depend on the noise level as $\lambda_{\text{ROF}} \approx 0.7\sigma$; for the large values of the noise (the last three entries) a better fit is $\lambda_{\text{ROF}} \approx 0.8\sigma$. We will use this approximation further in the initial calculation for the weight in the double-phase ROF model.

Noise variance σ^2	Optimal λ	Minimal LPIPS value
0.001	0.02	0.0820
0.002	0.03	0.1251
0.004	0.045	0.1785
0.007	0.065	0.2265
0.01	0.085	0.2599
0.02	0.12	0.3279
0.04	0.17	0.3979
0.07	0.22	0.4536
0.1	0.265	0.4873
0.2	0.315	0.5516

Table 2: Results of optimisation of λ using LPIPS

2.2 The adaptive double-phase ROF model

The second stage is the optimisation of the parameters λ , a and b in the double-phase ROF model, i.e.,

$$\min_u \int_{\Omega} |\nabla u| \, dx + \int_{\Omega} w(x) |\nabla u|^2 \, dx + \frac{1}{2\lambda} \int_{\Omega} |u - g|^2 \, dx,$$

with weight computed from the mollified output of classical ROF (using the already optimised parameter λ_{ROF}). More precisely, given the noisy datum g , let u_{ROF} denote the minimizer of the classical ROF model. We define its mollified version by

$$\tilde{u}_{\text{ROF}} = \rho_r * u_{\text{ROF}},$$

where r is a small mollification radius (we set $r = 2$ in these experiments). We use the weight w_1 (see the 1D supplement), defined by

$$w_1(x) = W_1(|\nabla \tilde{u}_{\text{ROF}}|(x)),$$

where the function W_1 is given by

$$W_1(x) = \max \left(0, a - b \max \left(x, \frac{a}{2b} \right) \right).$$

Again, λ corresponds in a non-explicit way to the noise level, while a and b are parameters for fine-tuning the model. Assuming that the noise is Gaussian with standard deviation σ , again λ increases as σ increases but no exact rate is given. We thus computed the average values of the SSIM and LPIPS metrics on the “train” part of the BSDS500 dataset and picked the optimal values of λ , a and b/a depending on the noise level. The results are presented in Tables 3 and 4 respectively.

Noise variance	Optimal λ	Optimal a	Optimal b/a	Maximal SSIM value
0.001	0.015	70	100	0.9153
0.002	0.025	20	80	0.8843
0.004	0.03	80	60	0.8439
0.007	0.045	70	50	0.8026
0.01	0.055	60	30	0.7715
0.02	0.08	60	20	0.7089
0.04	0.10	60	20	0.6368
0.07	0.155	60	40	0.5966

Table 3: Results of optimisation of λ , a and b/a using SSIM

Noise variance	Optimal λ	Optimal a	Optimal b/a	Minimal LPIPS value
0.001	0.0175	40	100	0.0816
0.002	0.025	30	80	0.1265
0.004	0.035	60	70	0.1742
0.007	0.05	60	40	0.2227
0.01	0.065	50	70	0.2541
0.02	0.085	60	20	0.3199
0.04	0.125	50	50	0.3884
0.07	0.15	50	10	0.4400

Table 4: Results of optimisation of λ , a and b/a using LPIPS

Let us describe in a bit more detail the dependence of the metrics SSIM and LPIPS with respect to the parameters a and b/a . It turns out that this dependence is less pronounced with respect to the dependence on λ ; for this reason, we believe it is feasible to use a simplified version of the double-phase ROF model, where the values of a and b/a are kept constant in a reasonable range and the only parameter to be optimised is λ . To illustrate this, let us present several detailed tables on the dependence of the SSIM and LPIPS metrics on these parameters (for the noise variance $\sigma^2 = 0.001$): these are Tables 5–14.

Table 5: Dependence on a and b/a for $\sigma^2 = 0.001$, average SSIM values: $\lambda = 0.01$

$\begin{matrix} \text{b/a} \\ \text{a} \end{matrix}$	50	60	70	80	90	100
10	0.8880	0.8874	0.8870	0.8865	0.8860	0.8852
20	0.8955	0.8952	0.8946	0.8938	0.8930	0.8920
30	0.9010	0.9005	0.8998	0.8988	0.8979	0.8968
40	0.9041	0.9043	0.9035	0.9027	0.9016	0.9003
50	0.9066	0.9068	0.9063	0.9055	0.9046	0.9032
60	0.9081	0.9087	0.9083	0.9077	0.9067	0.9054
70	0.9092	0.9100	0.9099	0.9093	0.9083	0.9071

Table 6: Dependence on a and b/a for $\sigma^2 = 0.001$, average SSIM values: $\lambda = 0.0125$

$\begin{smallmatrix} \text{b/a} \\ \text{a} \end{smallmatrix}$	50	60	70	80	90	100
10	0.9019	0.9016	0.9015	0.9013	0.9007	0.9003
20	0.9065	0.9067	0.9065	0.9060	0.9055	0.9048
30	0.9090	0.9094	0.9093	0.9091	0.9086	0.9080
40	0.9103	0.9111	0.9114	0.9112	0.9108	0.9103
50	0.9111	0.9122	0.9127	0.9127	0.9108	0.9117
60	0.9113	0.9129	0.9135	0.9137	0.9133	0.9128
70	0.9111	0.9130	0.9139	0.9143	0.9143	0.9136

Table 7: Dependence on a and b/a for $\sigma^2 = 0.001$, average SSIM values: $\lambda = 0.015$

$\begin{smallmatrix} \text{b/a} \\ \text{a} \end{smallmatrix}$	50	60	70	80	90	100
10	0.9094	0.9096	0.9097	0.9097	0.9092	0.9092
20	0.9112	0.9120	0.9121	0.9122	0.9120	0.9119
30	0.9116	0.9127	0.9133	0.9136	0.9135	0.9132
40	0.9112	0.9129	0.9138	0.9141	0.9145	0.9142
50	0.9106	0.9126	0.9138	0.9147	0.9148	0.9148
60	0.9097	0.9123	0.9138	0.9146	0.9150	0.9152
70	0.9088	0.9117	0.9133	0.9145	0.9152	0.9153

Table 8: Dependence on a and b/a for $\sigma^2 = 0.001$, average SSIM values: $\lambda = 0.0175$

$\begin{smallmatrix} \text{b/a} \\ \text{a} \end{smallmatrix}$	50	60	70	80	90	100
10	0.9119	0.9123	0.9123	0.9127	0.9127	0.9126
20	0.9111	0.9123	0.9131	0.9134	0.9136	0.9136
30	0.9100	0.9116	0.9129	0.9133	0.9136	0.9138
40	0.9085	0.9106	0.9123	0.9131	0.9138	0.9139
50	0.9070	0.9095	0.9114	0.9127	0.9134	0.9139
60	0.9054	0.9087	0.9108	0.9121	0.9132	0.9138
70	0.9039	0.9076	0.9099	0.9116	0.9127	0.9134

Table 9: Dependence on a and b/a for $\sigma^2 = 0.001$, average SSIM values: $\lambda = 0.02$

$\begin{smallmatrix} \text{b/a} \\ \text{a} \end{smallmatrix}$	50	60	70	80	90	100
10	0.9099	0.9107	0.9112	0.9115	0.9117	0.9117
20	0.9079	0.9092	0.9104	0.9110	0.9114	0.9117
30	0.9055	0.9075	0.9090	0.9100	0.9108	0.9114
40	0.9032	0.9061	0.9079	0.9091	0.9101	0.9107
50	0.9010	0.9045	0.9068	0.9083	0.9095	0.9105
60	0.8992	0.9030	0.9057	0.9076	0.9089	0.9098
70	0.8974	0.9016	0.9047	0.9066	0.9081	0.9093

Table 10: Dependence on a and b/a for $\sigma^2 = 0.001$, average LPIPS values: $\lambda = 0.01$

$\begin{smallmatrix} & b/a \\ a & \end{smallmatrix}$	50	60	70	80	90	100
10	0.1119	0.1123	0.1120	0.1125	0.1132	0.1140
20	0.1055	0.1055	0.1056	0.1066	0.1074	0.1079
30	0.1014	0.1010	0.1011	0.1023	0.1028	0.1039
40	0.0985	0.0980	0.0980	0.0985	0.0994	0.1007
50	0.0964	0.0955	0.0957	0.0963	0.0971	0.0985
60	0.0952	0.0940	0.0935	0.0940	0.0950	0.0961
70	0.0941	0.0924	0.0922	0.0924	0.0933	0.0947

Table 11: Dependence on a and b/a for $\sigma^2 = 0.001$, average LPIPS values: $\lambda = 0.0125$

$\begin{smallmatrix} & b/a \\ a & \end{smallmatrix}$	50	60	70	80	90	100
10	0.0986	0.0984	0.0984	0.0985	0.0992	0.0995
20	0.0942	0.0939	0.0938	0.0941	0.0943	0.0951
30	0.0921	0.0910	0.0906	0.0911	0.0916	0.0922
40	0.0908	0.0894	0.0889	0.0886	0.0892	0.0902
50	0.0902	0.0879	0.0871	0.0872	0.0877	0.0886
60	0.0900	0.0873	0.0859	0.0860	0.0861	0.0867
70	0.0900	0.0867	0.0853	0.0848	0.0850	0.0861

Table 12: Dependence on a and b/a for $\sigma^2 = 0.001$, average LPIPS values: $\lambda = 0.015$

$\begin{smallmatrix} & b/a \\ a & \end{smallmatrix}$	50	60	70	80	90	100
10	0.0893	0.0892	0.0888	0.0887	0.0892	0.0890
20	0.0877	0.0865	0.0862	0.0857	0.0859	0.0866
30	0.0871	0.0855	0.0843	0.0842	0.0843	0.0846
40	0.0873	0.0847	0.0836	0.0829	0.0829	0.0832
50	0.0880	0.0854	0.0834	0.0824	0.0825	0.0823
60	0.0893	0.0857	0.0835	0.0824	0.0820	0.0819
70	0.0906	0.0862	0.0840	0.0826	0.0817	0.0817

Table 13: Dependence on a and b/a for $\sigma^2 = 0.001$, average LPIPS values: $\lambda = 0.0175$

$\begin{smallmatrix} & b/a \\ a & \end{smallmatrix}$	50	60	70	80	90	100
10	0.0842	0.0836	0.0830	0.0827	0.0830	0.0830
20	0.0853	0.0835	0.0827	0.0819	0.0819	0.0819
30	0.0870	0.0849	0.0828	0.0820	0.0818	0.0817
40	0.0890	0.0860	0.0841	0.0828	0.0820	0.0816
50	0.0914	0.0878	0.0848	0.0832	0.0823	0.0818
60	0.0935	0.0892	0.0863	0.0842	0.0831	0.0823
70	0.0956	0.0909	0.0874	0.0849	0.0838	0.0827

Table 14: Dependence on a and b/a for $\sigma^2 = 0.001$, average LPIPS values: $\lambda = 0.02$

$\begin{array}{c} \text{b/a} \\ \text{a} \end{array}$	50	60	70	80	90	100
10	0.0850	0.0835	0.0828	0.0826	0.0821	0.0820
20	0.0879	0.0860	0.0846	0.0837	0.0831	0.0827
30	0.0915	0.0888	0.0865	0.0853	0.0844	0.0836
40	0.0951	0.0912	0.0887	0.0868	0.0855	0.0845
50	0.0982	0.0937	0.0905	0.0882	0.0868	0.0856
60	0.1012	0.0960	0.0924	0.0897	0.0879	0.0864
70	0.1038	0.0980	0.0938	0.0910	0.0890	0.0875

2.3 Optimisation of other tested models

In this section, we briefly present how we chose the parameters in other models that we used for comparison in the next section: in the Huber-ROF model [4], the total generalised variation [2], the Chen-Levine-Rao model [6] and the non-local means model [3]. In each case, we again computed the average values of the LPIPS, SSIM and PSNR metrics on the “train” part of the BSDS500 dataset and picked the optimal values of parameters depending on the noise level.

2.3.1 Huber-ROF model

We start with the Huber-ROF model [4]

$$\min_u \int_{\Omega} |\nabla u|_{\alpha} \, dx + \frac{1}{2\lambda} \int_{\Omega} |u - f|^2 \, dx,$$

where $|\cdot|_{\alpha}$ denotes the Huber function given by

$$|\xi|_{\alpha} = \begin{cases} \frac{1}{2\alpha} |\xi|^2 & \text{if } |\xi| \leq \alpha; \\ |\xi| - \frac{\alpha}{2} & \text{if } |\xi| > \alpha. \end{cases}$$

This model has two adjustable parameters: λ which plays a similar role as in the classical ROF model and the cutoff level α . We present the results of the optimisation below, using either the metric SSIM or LPIPS, in Tables 15 and 16 respectively.

Noise variance	Optimal λ	Optimal α	Maximal SSIM value
0.001	0.02	0.005	0.9185
0.002	0.035	0.005	0.8785
0.004	0.05	0.005	0.8348
0.007	0.06	0.005	0.7930
0.01	0.08	0.005	0.7639
0.02	0.12	0.005	0.7016
0.04	0.17	0.005	0.6369
0.07	0.23	0.005	0.5851

Table 15: Huber-ROF: results of optimisation of λ and α using SSIM

Noise variance	Optimal λ	Optimal α	Minimal LPIPS value
0.001	0.02	0.005	0.0834
0.002	0.035	0.005	0.1281
0.004	0.05	0.005	0.1787
0.007	0.06	0.005	0.2306
0.01	0.08	0.005	0.2601
0.02	0.12	0.005	0.3278
0.04	0.18	0.005	0.3968
0.07	0.23	0.005	0.4520

Table 16: Huber-ROF: results of optimisation of λ and α using LPIPS

2.3.2 Non-local means (NL-means) model

The second optimised model is the non-local means (NL-means) model introduced in [3]. It is a non-variational and non-local denoising model; we consider a variant which produces a denoised image by the explicit formula

$$u(x) = \frac{1}{Z(x)} \int_{S_a(x)} \exp(-(d(x, y)^2 - 2\sigma^2)_+/h^2\sigma^2) g(y) dy,$$

where

$$d(x, y) = \frac{1}{b^2} \int_{S_b(0)} |g(x+z) - g(y+z)|^2 dz,$$

$S_a(x)$ is the square of side length a centered at x , $a > 0$, $b > 0$, $h > 0$, $\sigma > 0$ are parameters, and

$$Z(x) = \int_{S_a(x)} \exp(-(d(x, y)^2 - 2\sigma^2)_+/h^2\sigma^2) dy$$

is a normalisation. It corresponds to the 'fast' setting of `denoise_nl_means` from the scikit-image library. This model has three adjustable parameters: the comparison patch (averaging area); the research block (context area); and the parameter h in the weight. The model is already optimised in [3], but it is done for images in the $[0, 255]$ range and due to nonlinearity of the model the parameter h cannot be rescaled in a simple fashion to the range $[0, 1]$ used in the implementation of the other models. Therefore, we used the averaging and context areas as given in [3] and optimised only the parameter h . We present the results of the optimisation below, using either the metric SSIM or LPIPS, in Tables 17 and 18 respectively.

We also optimised the version of the above model with $\sigma = 0$ (in the implementation: without supplying the value of sigma in the `denoise_nl_means` function). We present the results of the optimisation below, using either the metric SSIM or LPIPS, in Tables 19 and 20 respectively. In general, this version only produced (slightly) better results for very small noise levels, therefore in the comparisons in Section 3 we only use the main model (with σ).

Noise variance	Optimal h	Maximal SSIM value
0.001	0.6	0.9005
0.002	0.65	0.8527
0.004	0.45	0.8383
0.007	0.45	0.7872
0.01	0.5	0.7463
0.02	0.4	0.6689
0.04	0.35	0.5677
0.07	0.4	0.4477

Table 17: NL-means (with σ): results of optimisation of h using SSIM

Noise variance	Optimal h	Minimal LPIPS value
0.001	0.6	0.0848
0.002	0.65	0.1300
0.004	0.4	0.1382
0.007	0.45	0.1854
0.01	0.5	0.2220
0.02	0.4	0.3068
0.04	0.4	0.4191
0.07	0.45	0.5054

Table 18: NL-means (with σ): results of optimisation of h using LPIPS

Noise variance	Optimal h	Maximal SSIM value
0.001	0.65	0.9148
0.002	0.7	0.8816
0.004	0.75	0.8332
0.007	0.75	0.7849
0.01	0.75	0.7486
0.02	0.75	0.6610
0.04	0.75	0.5380
0.07	0.75	0.4253

Table 19: NL-means (without σ): results of optimisation of h using SSIM

Noise variance	Optimal h	Minimal LPIPS value
0.001	0.65	0.0784
0.002	0.7	0.1121
0.004	0.7	0.1591
0.007	0.75	0.2069
0.01	0.75	0.2419
0.02	0.75	0.3250
0.04	0.75	0.4305
0.07	0.75	0.5254

Table 20: NL-means (without σ): results of optimisation of h using LPIPS

2.3.3 The Chen-Levine-Rao variable exponent model

In this part we present the results for the Chen–Levine–Rao variable exponent ROF model [6]. The variable exponent is defined as

$$q(x) = 1 + \frac{1}{1 + K|\nabla G_\sigma * g(x)|^2},$$

where G_σ is a Gaussian kernel with variance $\sigma = 0.5$ and g is the noisy datum. The associated gradient flow is given by

$$\partial_t u = \operatorname{div}(\phi_r(x, \nabla u)) - \lambda(u - f),$$

where the integrand ϕ is defined by

$$\phi(x, r) = \begin{cases} \frac{1}{q(x)}|r|^{q(x)}, & |r| \leq \beta, \\ |r| - \frac{\beta q(x) - \beta^{q(x)}}{q(x)}, & |r| > \beta, \end{cases}$$

with $\beta > 0$ fixed. Following a slightly different approach from [6], we discretize the divergence of the flux

$$\operatorname{div}(\phi_r(x, \nabla u))$$

by treating separately the different components of ϕ_r , instead of expanding the full divergence as in [6]. More precisely, the discretization is performed in a split fashion taking into account the two different parts of ϕ_r : when $|\nabla u| \leq \beta$, (i.e. the superlinear growth part of the domain), the flux is approximated using standard finite differences (central differences for the gradient and backward differences for the divergence). As for $|\nabla u| > \beta$, (i.e. the linear growth term), we approximate the 1-Laplacian using the minmod scheme, as in [7]. We optimize the model for images in the range $[0, 255]$, using the parameter values K and β similar as in [6], while we also tune different values of the parameter $\lambda > 0$ across different noise levels. The results are summarized in Tables 21 and 22.

Noise variance	Optimal λ	Optimal K	Optimal β	Maximal SSIM value
65	0.3125	0.01	30.0	0.9183
130	0.1625	0.01	30.0	0.8784
261	0.125	0.01	30.0	0.8330
456	0.0875	0.005	100.0	0.7878
650	0.0625	0.01	30.0	0.7603
1301	0.0375	0.01	30.0	0.6927
2601	0.0275	0.01	30.0	0.6331
4552	0.025	0.01	30.0	0.4674

Table 21: Results of optimisation of λ , K and β using SSIM for Chen–Levine–Rao model

Noise variance	Optimal λ	Optimal K	Optimal β	Minimal LPIPS value
65	0.3125	0.01	30.0	0.0826
130	0.1625	0.01	30.0	0.1281
261	0.1125	0.01	30.0	0.1800
456	0.0875	0.005	100.0	0.2311
650	0.0625	0.01	30.0	0.2598
1301	0.0375	0.01	30.0	0.3306
2601	0.0275	0.01	30.0	0.4004
4552	0.025	0.01	30.0	0.5598

Table 22: Results of optimisation of λ , K and β using LPIPS for Chen–Levine–Rao model

2.3.4 The second-order total generalized variation

Another model that we consider is that based on the second-order total generalized variation (TGV $^2_\alpha$) regularizer [2]. The denoising problem is formulated as

$$\min_u \text{TGV}^2_\alpha(u) + \frac{1}{2} \int_\Omega |u - g|^2 dx, \quad (1)$$

where TGV^2_α can be defined as the solution of the minimization problem

$$\text{TGV}^2_\alpha(u) = \min_v \alpha_1 \int_\Omega |\nabla u - v| dx + \alpha_0 \int_\Omega |E(v)| dx,$$

where the minimum is taken over all vector fields v on Ω , and

$$E(v) = \frac{1}{2}(\nabla v + \nabla v^T)$$

denotes the symmetrized gradient. Here, the auxiliary variable v approximates the gradient ∇u , i.e., the first term plays a similar role to the total variation regularizer. Then, the second term penalizes variations of v , thus controlling changes of the gradient. The positive parameters α_0, α_1 balance the effect of the regularizer against the fidelity term. In this way, TGV^2_α extends the classical total variation by allowing piecewise smooth (rather than piecewise constant) solutions, reducing the staircasing effect and provides more natural reconstruction of the image. The minimization problem (1) can be equivalently reformulated, after suitable

transformations, as a convex-concave saddle-point problem. This reformulation enables the use of the primal-dual algorithm of Chambolle–Pock [5], where the corresponding resolvent operators admit closed form expressions in terms of projections. For a detailed derivation of this formulation and the corresponding algorithm, we refer the interested reader to [1].

We tested the effectiveness of this model on the training set of images (BSDS500) across different noise levels. We report the corresponding averaged optimum values of the SSIM and LPIPS scores in the following tables as well as the optimal values of the parameters that we obtained.

Noise variance	Optimal α_1	Optimal α_0	Maximal SSIM value
0.001	0.02	0.4	0.9190
0.002	0.03	0.02	0.8857
0.004	0.04	0.05	0.8361
0.007	0.05	0.04	0.7806
0.01	0.075	0.15	0.7632
0.02	0.1	0.5	0.6910
0.04	0.15	0.15	0.6340
0.07	0.2	0.2	0.5832

Table 23: Results of optimisation of α_1 and α_0 using SSIM for TGV_α^2

Noise variance	Optimal α_1	Optimal α_0	Minimal LPIPS value
0.001	0.02	0.4	0.0811
0.002	0.03	0.03	0.1170
0.004	0.05	0.1	0.1732
0.007	0.05	0.04	0.2351
0.01	0.075	0.15	0.2624
0.02	0.1	0.5	0.3383
0.04	0.15	0.2	0.4028
0.07	0.2	0.35	0.4580

Table 24: Results of optimisation of α_1 and α_0 using LPIPS for TGV_α^2

3 Comparison between the models

In this section we compare the test results between the adaptive double-phase ROF model, the techniques optimised in the previous section, and several variants of the double-phase ROF model described below. First let us state what we consider to be the main version of the double-phase ROF model used for comparison. The setup is as follows: we first run the calculation for the classical ROF model

$$\min_u \int |\nabla u| \, dx + \frac{1}{2\lambda_{\text{ROF}}} \int |u - g|^2 \, dx.$$

From the solution u_{ROF} we compute the weight to be used by setting

$$w(x) = W(|\nabla \tilde{u}_{\text{ROF}}|(x)),$$

where $\tilde{u}_{\text{ROF}} = \rho_r * u_{\text{ROF}}$ is the mollified version of u_{ROF} obtained by mollification with a radial kernel with a small radius r and the function W is given by

$$W(x) = \max \left(0, a - b \max \left(x, \frac{a}{2b} \right) \right).$$

Therefore, in order to compute the solution to the double-phase ROF model (dpROF)

$$\min_u \int_{\Omega} |\nabla u| \, dx + \int_{\Omega} w(x) |\nabla u|^2 \, dx + \frac{1}{2\lambda} \int_{\Omega} |u - g|^2 \, dx,$$

we need to pick the values of λ_{ROF} in the Rudin–Osher–Fatemi model, λ to be used in the double-phase ROF model, and the weight parameters a and b . We also need to pick the mollification radius (always set to $r = 2$) and the prescribed accuracy levels ε_{ROF} (i.e., the algorithm stops if the L^2 distance between two iterations in a row is lower than this value) and $\varepsilon_{\text{dpROF}}$ for the classical and double-phase ROF models respectively.

Table 25: Choice of parameters in the adaptive double-phase ROF model

Noise level	Value of λ_{ROF}	Value of λ	Value of a	Value of b/a
Small ($\sigma^2 \leq 0.005$)	$0.7 \cdot \sigma$	$0.5 \cdot \sigma$	60	100
Medium ($0.005 < \sigma^2 \leq 0.015$)				50
Large ($\sigma^2 > 0.015$)	$0.8 \cdot \sigma$			25

We observe that the optimisation of the classical ROF model with respect to λ_{ROF} (in Section 2.1) gave the following results: for small and medium noise levels, $\lambda_{\text{ROF}} \approx 0.7\sigma$ was a good fit, while for large noise levels $\lambda_{\text{ROF}} \approx 0.8\sigma$ was a good fit. We will use these parameters in the initial ROF calculation. Furthermore, the optimisation of the double-phase ROF model (in Section 2.2) gave the results that for all noise levels $\lambda \approx 0.5\sigma$ was a good fit; we will use this parameter in the double-phase ROF calculation. This is shown in the second and third columns of Table 25.

The next question concerns the choice of parameters a and b . The optimisation results in Section 2.2 show that, in general, there is only little variance in the results for different choices of a and b . More importantly, for a the trend line is nearly constant, while for b it is decreasing; to have a simple choice of parameters, we propose to take as a the median value of all optimal choices of a for different noise levels, i.e., $a = 60$. For b (or equivalently: b/a , which was the optimised parameter) the median value is $b/a = 50$, yet we also observe that the optimal b/a visibly decreases as the noise variance increases. Therefore, as a correction, we introduced the following modification: for small values of the noise we set $b/a = 100$, for medium levels we take $b/a = 50$, and use the value $b/a = 25$ for large noise levels. This is shown in the last two columns of Table 25.

Finally, regarding the accuracy levels ε_{ROF} and $\varepsilon_{\text{dpROF}}$: we conjectured that in order to reduce the computational cost, it is enough to use the initial ROF computation with reduced accuracy level. It turns out that the results, both visually and in the calculated metrics, there is virtually no difference with even if one reduces the initial ROF accuracy by two-three orders

of magnitude; the results for $\sigma^2 = 0.004$ and $\sigma^2 = 0.04$ (and accuracy levels $\varepsilon = 10^{-4}$ and $\varepsilon = 10^{-6}$) are shown in Tables 42-43 and 44-45 respectively. Therefore, in the main version of the double-phase ROF model we use $\varepsilon_{\text{ROF}} = \sqrt{\varepsilon_{\text{dpROF}}}$.

We finally compare the performance of the models listed (and briefly described if necessary) below. All the models for the comparison are considered for the same accuracy level (even the classical ROF model; then the initial calculation with reduced accuracy is only used for the calculation in dpROF). The comparison is done by performing a single run of all the models over the “test” part of the BSDS500 dataset.

1. dpROF: the adaptive double-phase ROF model with parameters as given in Table 25.
2. ROF: the classical Rudin–Osher–Fatemi model, with the parameter λ as optimised in Section 2.1.
3. TGV: the total generalised variation model, with parameters α_1, α_2 as optimised in Section 2.3.4.
4. NL-means: the non-local means model, with h as optimised in Section 2.3.2.
5. The Chen–Levine–Rao model, with λ, K and β as optimised in Section 2.3.3.
6. The Huber–ROF model, with parameters λ and α as optimised in Section 2.3.1.
7. dpROF-noisy: The double-phase ROF model with the weight computed directly from the noisy datum. More precisely, in the definition of the weight in the dpROF model, instead of setting

$$w(x) = W(|\nabla \tilde{u}_{\text{ROF}}|(x))$$

we simply use

$$w(x) = W(|\nabla \tilde{g}|(x)),$$

i.e., we use mollified noisy datum. We then use the same parameters as in the main version of dpROF.

8. dpROF-SSIM: The dpROF model, but instead of using simplified ways of choosing parameters $\lambda_{\text{ROF}}, \lambda, a$ and b/a we use the exact optimal values for SSIM from the tables in Sections 2.1 and 2.2.
9. dpROF-LPIPS: The dpROF model, but instead of using simplified ways of choosing parameters $\lambda_{\text{ROF}}, \lambda, a$ and b/a we use the exact optimal values for LPIPS from the tables in Sections 2.1 and 2.2.
10. dpROF-edge: The double-phase ROF model with the weight computed from an edge map obtained from the noisy datum using a standard edge-detection algorithm. In the results in the table we used the Canny edge detection (in the initial tests it proved superior to Sobel/Scharr edge detection for this purpose). We then use the same parameters as in the main version of dpROF.

The results are shown in Tables 26-41; the dependency of the metrics on the noise level are shown for the best-performing methods, i.e., dpROF, ROF, TGV and NL-means also visually in Figures 1-5. We show, for two accuracy levels $\varepsilon = 10^{-4}$ and $\varepsilon = 10^{-6}$, the average

computation times and the average SSIM, LPIPS and PSNR metrics. All the computations were done locally on the same device to ensure that the times are comparable. The results show that the dpROF model, even in the mainline simplified version, delivers promising results which are comparable or better to all other tested methods, even at the 10^{-4} accuracy level; this value is also included in the second half of comparison tables (for the 10^{-6} accuracy level). This allows for a significant reduction in computation time with a small loss in accuracy of the reconstruction. Furthermore, the comparison with dpROF-SSIM and dpROF-LPIPS show that while there is some more space to optimise the model, the baseline version performs really well and the difference is not too large (with the simplified model being even better on some occasions). Finally, the comparisons with dpROF-noisy and dpROF-edge show that, at least for large noise levels, an initial edge detection based on the ROF method leads to better results than other tested (non-variational) methods of constructing the weight.

Table 26: Comparison of different denoising methods: $\sigma^2 = 0.001$, $\varepsilon = 10^{-4}$

Method	Metrics			
	Avg. time	Avg. SSIM $\uparrow$	Avg. LPIPS $\downarrow$	Avg. PSNR $\uparrow$
dpROF	0.05s + 0.87s	0.9206	0.0823	33.32
ROF	0.23s	0.9179	0.0832	33.18
TGV	7.36s	0.9184	0.0823	33.20
NL-means	1.05s	0.9034	0.0898	32.41
Chen-Levine-Rao	0.48s	0.9183	0.0809	33.32
Huber-ROF	0.62s	0.9180	0.0841	33.20
dpROF-noisy	1.06s	0.9182	0.0832	33.15
dpROF-SSIM	0.05s + 0.90s	0.9208	0.0826	33.35
dpROF-LPIPS	0.09s + 0.97s	0.9193	0.0828	33.20
dpROF-edge	1.94s	0.9099	0.0950	31.97

Table 27: Comparison of different denoising methods: $\sigma^2 = 0.002$, $\varepsilon = 10^{-4}$

Method	Metrics			
	Avg. time	Avg. SSIM $\uparrow$	Avg. LPIPS $\downarrow$	Avg. PSNR $\uparrow$
dpROF	0.06s + 0.99s	0.8871	0.1240	31.45
ROF	0.27s	0.8823	0.1250	31.26
TGV	2.37s	0.8867	0.1131	31.37
NL-means	1.07s	0.8592	0.1367	30.38
Chen-Levine-Rao	1.24s	0.8789	0.1249	31.18
Huber-ROF	0.79s	0.8788	0.1266	31.00
dpROF-noisy	1.17s	0.8829	0.1262	31.32
dpROF-SSIM	0.06s + 1.09s	0.8853	0.1249	31.42
dpROF-LPIPS	0.09s + 1.09s	0.8861	0.1236	31.40
dpROF-edge	2.22s	0.8721	0.1431	30.33

Acknowledgments. The work of the first author has been supported by the Austrian

Table 28: Comparison of different denoising methods: $\sigma^2 = 0.004$, $\varepsilon = 10^{-4}$

Method	Metrics			
	Avg. time	Avg. SSIM $\uparrow$	Avg. LPIPS $\downarrow$	Avg. PSNR $\uparrow$
dpROF	0.06s + 1.10s	0.8411	0.1815	29.61
ROF	0.33s	0.8366	0.1777	29.44
TGV	4.93s	0.8371	0.1742	29.60
NL-means	0.95s	0.8407	0.1385	29.65
Chen-Levine-Rao	3.25s	0.8346	0.1778	29.52
Huber-ROF	0.98s	0.8347	0.1775	29.30
dpROF-noisy	1.23s	0.8340	0.1835	29.52
dpROF-SSIM	0.06s + 1.04s	0.8432	0.1769	29.58
dpROF-LPIPS	0.10s + 1.21s	0.8433	0.1746	29.59
dpROF-edge	2.51s	0.8233	0.2026	28.76

Table 29: Comparison of different denoising methods: $\sigma^2 = 0.007$, $\varepsilon = 10^{-4}$

Method	Metrics			
	Avg. time	Avg. SSIM $\uparrow$	Avg. LPIPS $\downarrow$	Avg. PSNR $\uparrow$
dpROF	0.07s + 1.18s	0.8014	0.2263	28.23
ROF	0.37s	0.7922	0.2289	28.10
TGV	2.74s	0.7833	0.2310	28.16
NL-means	0.99s	0.7903	0.1874	28.17
Chen-Levine-Rao	2.05s	0.7887	0.2284	28.06
Huber-ROF	1.06s	0.7929	0.2290	28.12
dpROF-noisy	1.35s	0.7848	0.2376	28.10
dpROF-SSIM	0.07s + 1.27s	0.8007	0.2246	28.20
dpROF-LPIPS	0.12s + 1.45s	0.7914	0.2299	28.02
dpROF-edge	2.78s	0.7751	0.2581	27.53

Science Fund (FWF), grants 10.55776/ESP88 and 10.55776/I5149. The second author has been partially supported by the grant 2024/55/D/ST1/03055 of the National Science Centre (NCN), Poland. The third author has been partially supported by the Special Account for Research Funding of the National Technical University of Athens.

Table 30: Comparison of different denoising methods: $\sigma^2 = 0.01$, $\varepsilon = 10^{-4}$

Method	Metrics			
	Avg. time	Avg. SSIM $\uparrow$	Avg. LPIPS $\downarrow$	Avg. PSNR $\uparrow$
dpROF	0.07s + 1.26s	0.7720	0.2618	27.38
ROF	0.42s	0.7619	0.2615	27.23
TGV	6.74s	0.7651	0.2559	27.27
NL-means	0.99s	0.7527	0.2262	27.21
Chen-Levine-Rao	8.25s	0.7584	0.2600	27.11
Huber-ROF	1.22s	0.7628	0.2591	27.19
dpROF-noisy	1.50s	0.7472	0.2758	27.20
dpROF-SSIM	0.12s + 1.43s	0.7639	0.2605	27.25
dpROF-LPIPS	0.13s + 1.79s	0.7698	0.2569	27.33
dpROF-edge	3.14s	0.7404	0.2960	26.75

Table 31: Comparison of different denoising methods: $\sigma^2 = 0.02$, $\varepsilon = 10^{-4}$

Method	Metrics			
	Avg. time	Avg. SSIM $\uparrow$	Avg. LPIPS $\downarrow$	Avg. PSNR $\uparrow$
dpROF	0.08s + 1.43s	0.7025	0.3274	25.74
ROF	0.56s	0.6959	0.3289	25.55
TGV	7.63s	0.6924	0.3377	25.67
NL-means	2.83s	0.6784	0.3122	25.30
Chen-Levine-Rao	8.36s	0.6889	0.3334	25.38
Huber-ROF	1.41s	0.6991	0.3276	25.58
dpROF-noisy	1.69s	0.6691	0.3532	25.47
dpROF-SSIM	0.08s + 2.32s	0.6992	0.3293	25.60
dpROF-LPIPS	0.15s + 2.52s	0.6948	0.3316	25.53
dpROF-edge	3.57s	0.6663	0.3723	25.23

Table 32: Comparison of different denoising methods: $\sigma^2 = 0.04$, $\varepsilon = 10^{-4}$

Method	Metrics			
	Avg. time	Avg. SSIM $\uparrow$	Avg. LPIPS $\downarrow$	Avg. PSNR $\uparrow$
dpROF	0.09s + 1.67s	0.6412	0.3910	24.20
ROF	0.64s	0.6309	0.3993	24.00
TGV	6.95s	0.6337	0.4031	24.13
NL-means	2.80s	0.5899	0.4283	23.36
Chen-Levine-Rao	4.32s	0.5810	0.5213	22.73
Huber-ROF	1.72s	0.6339	0.3968	24.00
dpROF-noisy	1.96s	0.5827	0.4340	23.70
dpROF-SSIM	0.09s + 2.02s	0.6432	0.4057	24.12
dpROF-LPIPS	0.17s + 2.37s	0.6443	0.3958	24.18
dpROF-edge	4.31s	0.5900	0.4454	23.67

Table 33: Comparison of different denoising methods: $\sigma^2 = 0.07$, $\varepsilon = 10^{-4}$

Method	Metrics			
	Avg. time	Avg. SSIM $\uparrow$	Avg. LPIPS $\downarrow$	Avg. PSNR $\uparrow$
dpROF	0.10s + 1.99s	0.5892	0.4441	22.77
ROF	0.74s	0.5781	0.4553	22.57
TGV	7.86s	0.5822	0.4633	22.71
NL-means	2.85s	0.5225	0.5171	21.87
Chen-Levine-Rao	11.90s	0.5516	0.5257	22.00
Huber-ROF	1.87s	0.5811	0.4525	22.53
dpROF-noisy	2.27s	0.5219	0.4909	22.30
dpROF-SSIM	0.11s + 2.50s	0.5920	0.4499	22.75
dpROF-LPIPS	0.19s + 3.06s	0.5883	0.4485	22.68
dpROF-edge	5.10s	0.5360	0.4969	22.36

Table 34: Comparison of different denoising methods: $\sigma^2 = 0.001$, $\varepsilon = 10^{-6}$

Method	Metrics			
	Avg. time	Avg. SSIM $\uparrow$	Avg. LPIPS $\downarrow$	Avg. PSNR $\uparrow$
dpROF	0.09s + 2.92s	0.9205	0.0824	33.32
dpROF ($\varepsilon = 10^{-4}$)	0.05s + 0.87s	0.9206	0.0823	33.32
ROF	0.77s	0.9179	0.0832	33.18
TGV	7.38s	0.9184	0.0823	33.20
NL-means	1.05s	0.9034	0.0898	32.41
Chen-Levine-Rao	8.37s	0.9156	0.0842	33.12
Huber-ROF	6.19s	0.9180	0.0841	33.20
dpROF-noisy	3.47s	0.9185	0.0845	33.27
dpROF-SSIM	0.14s + 2.92s	0.9208	0.0826	33.34
dpROF-LPIPS	0.14s + 3.32s	0.9201	0.0820	33.27
dpROF-edge	1.80s	0.9129	0.0931	32.47

Table 35: Comparison of different denoising methods: $\sigma^2 = 0.002$, $\varepsilon = 10^{-6}$

Method	Metrics			
	Avg. time	Avg. SSIM $\uparrow$	Avg. LPIPS $\downarrow$	Avg. PSNR $\uparrow$
dpROF	0.10s + 3.31s	0.8872	0.1238	31.45
dpROF ($\varepsilon = 10^{-4}$)	0.06s + 0.99s	0.8871	0.1240	31.45
ROF	1.00s	0.8823	0.1250	31.26
TGV	9.01s	0.8868	0.1131	31.37
NL-means	1.07s	0.8592	0.1367	30.38
Chen-Levine-Rao	8.14s	0.8727	0.1318	30.89
Huber-ROF	7.41s	0.8787	0.1267	30.99
dpROF-noisy	4.26s	0.8817	0.1289	31.37
dpROF-SSIM	0.10s + 4.15s	0.8854	0.1248	31.42
dpROF-LPIPS	0.16s + 4.14s	0.8860	0.1236	31.40
dpROF-edge	2.07s	0.8748	0.1398	30.83

Table 36: Comparison of different denoising methods: $\sigma^2 = 0.004$, $\varepsilon = 10^{-6}$

Method	Metrics			
	Avg. time	Avg. SSIM $\uparrow$	Avg. LPIPS $\downarrow$	Avg. PSNR $\uparrow$
dpROF	0.11s + 3.91s	0.8415	0.1808	29.61
dpROF ($\varepsilon = 10^{-4}$)	0.06s + 1.08s	0.8411	0.1815	29.61
ROF	1.27s	0.8366	0.1777	29.44
TGV	8.98s	0.8364	0.1678	29.31
NL-means	0.95s	0.8407	0.1385	29.65
Chen-Levine-Rao	8.92s	0.8357	0.1763	29.47
Huber-ROF	9.51s	0.8345	0.1776	29.29
dpROF-noisy	4.83s	0.8325	0.1856	29.52
dpROF-SSIM	0.11s + 3.92s	0.8430	0.1771	29.58
dpROF-LPIPS	0.15s + 4.79s	0.8432	0.1746	29.58
dpROF-edge	2.32s	0.8189	0.2075	28.79

Table 37: Comparison of different denoising methods: $\sigma^2 = 0.007$, $\varepsilon = 10^{-6}$

Method	Metrics			
	Avg. time	Avg. SSIM $\uparrow$	Avg. LPIPS $\downarrow$	Avg. PSNR $\uparrow$
dpROF	0.12s + 4.65s	0.8013	0.2262	28.23
dpROF ($\varepsilon = 10^{-4}$)	0.07s + 1.18s	0.8014	0.2263	28.23
ROF	1.59s	0.7915	0.2265	28.00
TGV	8.86s	0.7820	0.2335	28.14
NL-means	0.99s	0.7903	0.1874	28.17
Chen-Levine-Rao	7.98s	0.7850	0.2305	27.86
Huber-ROF	10.15s	0.7930	0.2289	28.12
dpROF-noisy	6.61s	0.7933	0.2303	28.13
dpROF-SSIM	0.12s + 5.24s	0.8003	0.2248	28.19
dpROF-LPIPS	0.18s + 5.92s	0.7913	0.2301	28.02
dpROF-edge	2.86s	0.7747	0.2584	27.53

Table 38: Comparison of different denoising methods: $\sigma^2 = 0.01$, $\varepsilon = 10^{-6}$

Method	Metrics			
	Avg. time	Avg. SSIM $\uparrow$	Avg. LPIPS $\downarrow$	Avg. PSNR $\uparrow$
dpROF	0.13s + 5.02s	0.7721	0.2615	27.38
dpROF ($\varepsilon = 10^{-4}$)	0.07s + 1.26s	0.7720	0.2618	27.38
ROF	1.92s	0.7570	0.2604	27.05
TGV	8.84s	0.7651	0.2559	27.27
NL-means	0.99s	0.7527	0.2262	27.21
Chen-Levine-Rao	8.73s	0.7584	0.2601	27.11
Huber-ROF	11.60s	0.7627	0.2591	27.18
dpROF-noisy	6.83s	0.7575	0.2679	27.27
dpROF-SSIM	0.13s + 7.52s	0.7706	0.2593	27.35
dpROF-LPIPS	0.22s + 7.38s	0.7697	0.2569	27.33
dpROF-edge	3.36s	0.7485	0.2875	26.81

Table 39: Comparison of different denoising methods: $\sigma^2 = 0.02$, $\varepsilon = 10^{-6}$

Method	Metrics			
	Avg. time	Avg. SSIM $\uparrow$	Avg. LPIPS $\downarrow$	Avg. PSNR $\uparrow$
dpROF	0.16s + 5.42s	0.7022	0.3278	25.73
dpROF ($\varepsilon = 10^{-4}$)	0.08s + 1.43s	0.7025	0.3274	25.74
ROF	2.33s	0.6959	0.3289	25.55
TGV	7.58s	0.6924	0.3377	25.67
NL-means	2.83s	0.6784	0.3122	25.30
Chen-Levine-Rao	16.37s	0.6889	0.3334	25.38
Huber-ROF	13.71s	0.6990	0.3276	25.58
dpROF-noisy	10.46s	0.6990	0.3285	25.63
dpROF-SSIM	0.16s + 10.09s	0.6988	0.3296	25.60
dpROF-LPIPS	0.26s + 10.64s	0.6947	0.3316	25.53
dpROF-edge	4.99s	0.6874	0.3502	25.24

Table 40: Comparison of different denoising methods: $\sigma^2 = 0.04$, $\varepsilon = 10^{-6}$

Method	Metrics			
	Avg. time	Avg. SSIM $\uparrow$	Avg. LPIPS $\downarrow$	Avg. PSNR $\uparrow$
dpROF	0.19s + 6.81s	0.6410	0.3911	24.19
dpROF ($\varepsilon = 10^{-4}$)	0.09s + 1.67s	0.6412	0.3910	24.20
ROF	2.76s	0.6303	0.3986	23.96
TGV	8.58s	0.6338	0.4029	24.13
NL-means	2.80s	0.5899	0.4283	23.36
Chen-Levine-Rao	8.17s	0.6287	0.4033	23.99
Huber-ROF	15.97s	0.6338	0.3968	24.00
dpROF-noisy	9.16s	0.5967	0.4254	23.84
dpROF-SSIM	0.19s + 8.42s	0.6433	0.4055	24.13
dpROF-LPIPS	0.32s + 9.90s	0.6443	0.3958	24.18
dpROF-edge	4.57s	0.5993	0.4383	23.76

Table 41: Comparison of different denoising methods: $\sigma^2 = 0.07$, $\varepsilon = 10^{-6}$

Method	Metrics			
	Avg. time	Avg. SSIM $\uparrow$	Avg. LPIPS $\downarrow$	Avg. PSNR $\uparrow$
dpROF	0.22s + 7.71s	0.5891	0.4442	22.77
dpROF ($\varepsilon = 10^{-4}$)	0.10s + 1.99s	0.5892	0.4441	22.77
ROF	3.17s	0.5780	0.4550	22.54
TGV	7.86s	0.5788	0.4572	22.67
NL-means	2.85s	0.5225	0.5171	21.87
Chen-Levine-Rao	45.53s	0.5605	0.4693	22.53
Huber-ROF	18.70s	0.5809	0.4526	22.52
dpROF-noisy	12.29s	0.5618	0.4665	22.59
dpROF-SSIM	0.22s + 10.31s	0.5919	0.4497	22.75
dpROF-LPIPS	0.36s + 12.67s	0.5883	0.4485	22.68
dpROF-edge	6.03s	0.5634	0.4759	22.51

Figure 1: Comparison of ROF, dpROF, TGV, NL-means, Chen-Levine-Rao and Huber-ROF models across different noise levels (SSIM)

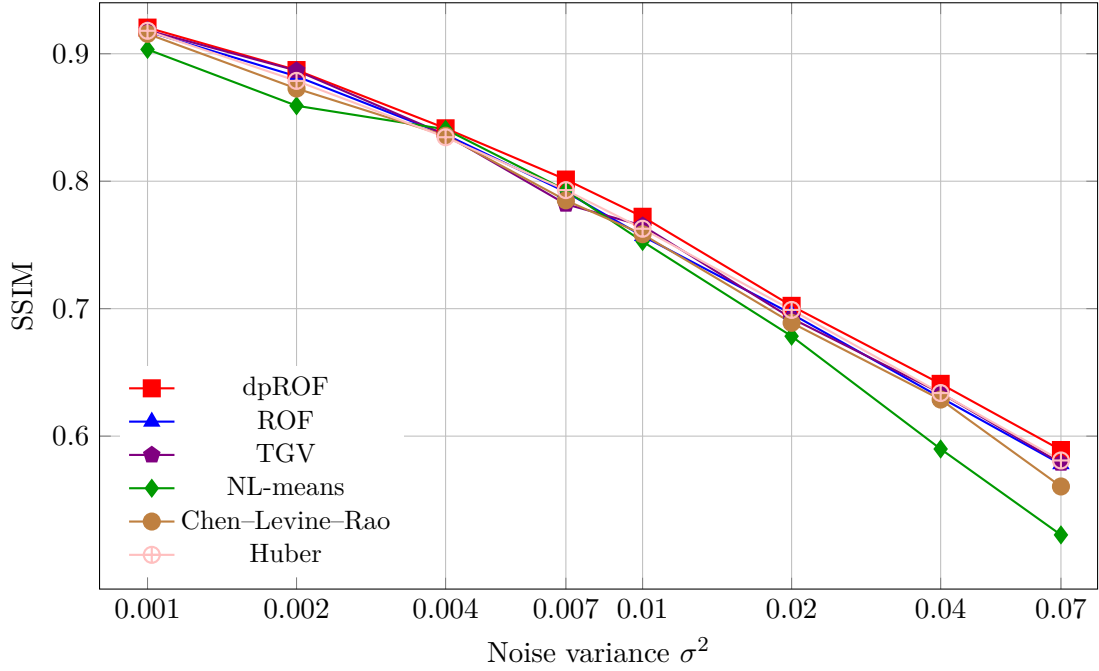


Figure 2: Comparison of ROF, dpROF, TGV, NL-means, Chen-Levine-Rao and Huber-ROF models across different noise levels (SSIM): zoomed-in first half

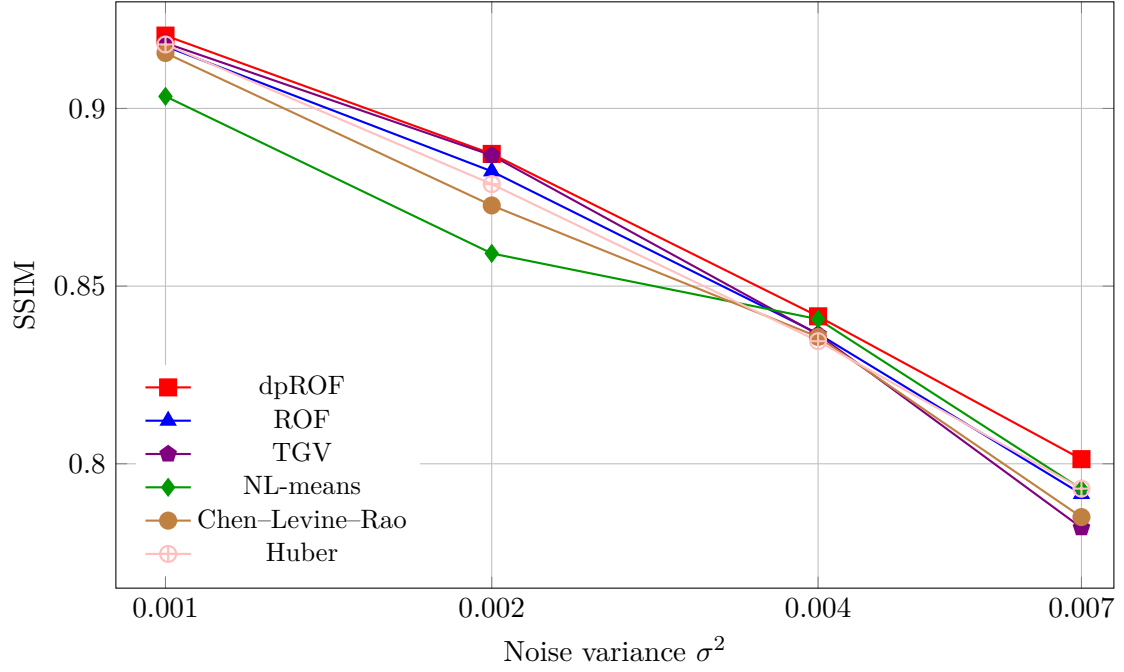


Figure 3: Comparison of ROF, dpROF, TGV, NL-means, Chen-Levine-Rao and Huber-ROF models across different noise levels (SSIM): zoomed-in second half

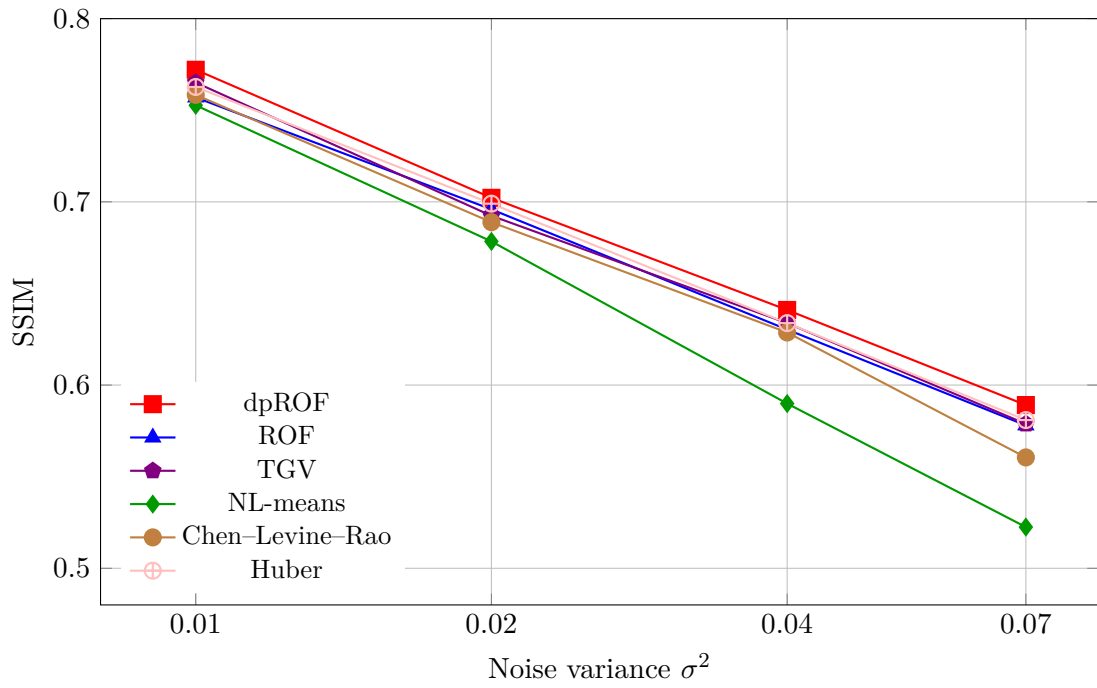


Figure 4: Comparison of ROF, dpROF, TGV, NL-means, Chen-Levine-Rao and Huber-ROF models across different noise levels (LPIPS)

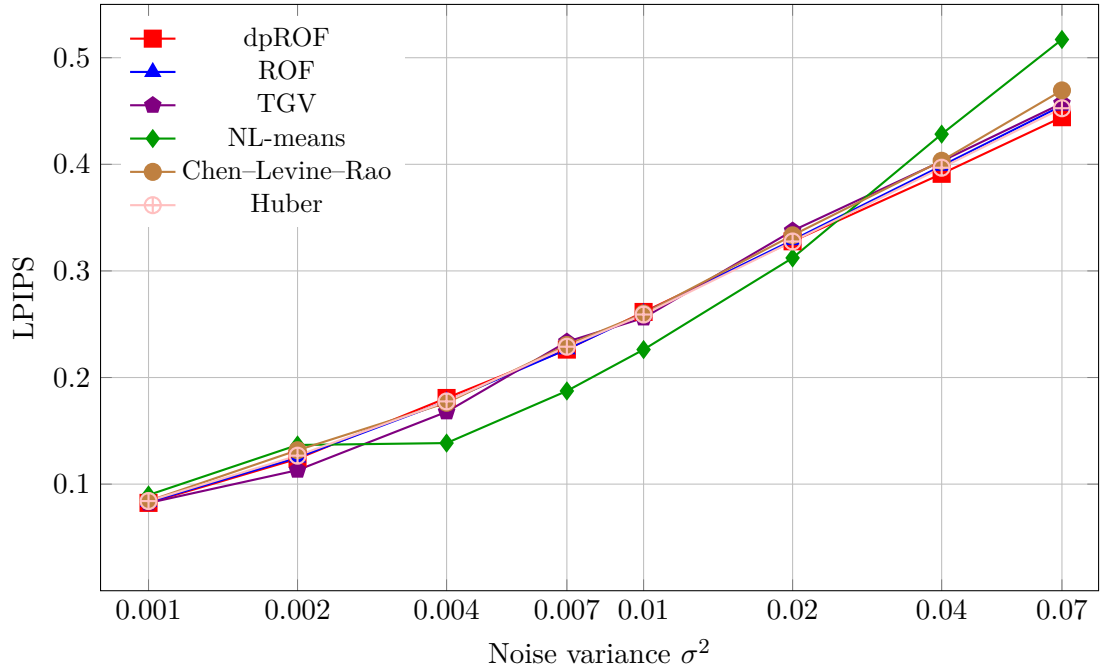


Figure 5: Comparison of ROF, dpROF, TGV, NL-means, Chen-Levine-Rao and Huber-ROF models across different noise levels (PSNR)

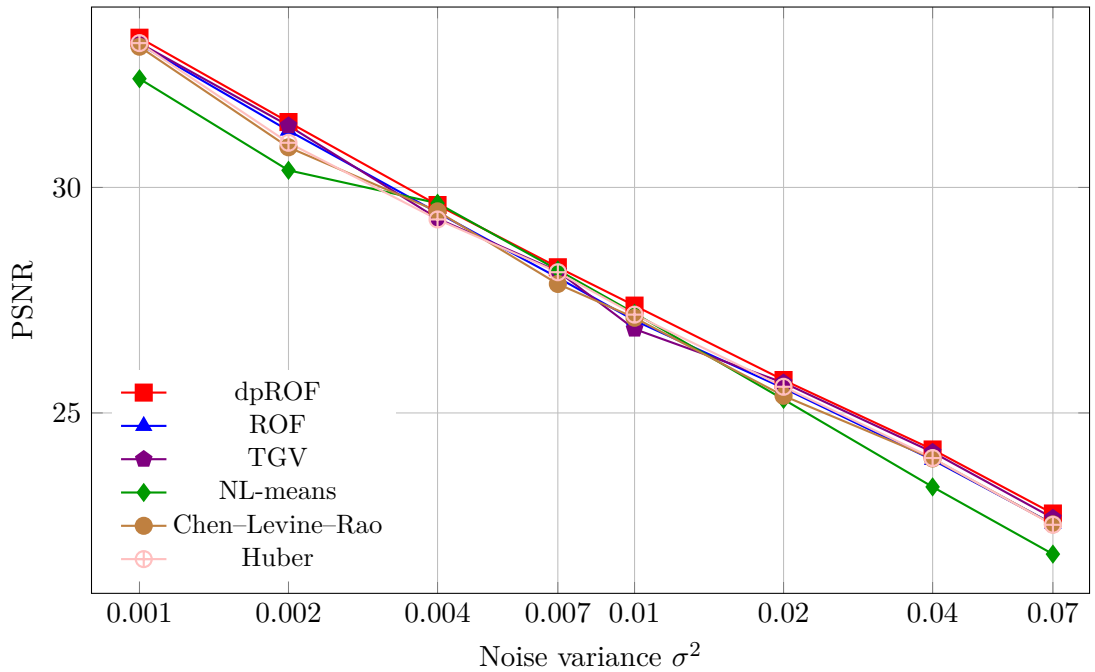


Table 42: Different accuracy levels for the initial ROF calculation, $\sigma^2 = 0.004$, $\varepsilon = 10^{-4}$

Compared metric	Change in ROF accuracy	Value
Time	dpROF	0.32s + 1.28s
	dpROF (x10)	0.16s + 1.30s
	dpROF (x50)	0.11s + 1.30s
	dpROF (x100)	0.09s + 1.31s
	dpROF (x500)	0.09s + 1.31s
	dpROF (x1000)	0.06s + 1.32s
LPIPS(dpROF, ·)	dpROF (x10)	1.7×10^{-7}
	dpROF (x50)	3.0×10^{-6}
	dpROF (x100)	9.4×10^{-6}
	dpROF (x500)	0.000125
	dpROF (x1000)	0.000358
SSIM(dpROF, ·)	dpROF (x10)	$1 - 1.0 \times 10^{-7}$
	dpROF (x50)	0.999998
	dpROF (x100)	0.999995
	dpROF (x500)	0.999940
	dpROF (x1000)	0.999832
PSNR(dpROF, ·)	dpROF (x10)	97.02
	dpROF (x50)	84.50
	dpROF (x100)	79.48
	dpROF (x500)	68.39
	dpROF (x1000)	63.89
LPIPS(image, ·)	dpROF	0.175058
	dpROF (x10)	0.175068
	dpROF (x50)	0.175105
	dpROF (x100)	0.175148
	dpROF (x500)	0.175460
	dpROF (x1000)	0.175813
SSIM(image, ·)	dpROF	0.843122
	dpROF (x10)	0.843121
	dpROF (x50)	0.843114
	dpROF (x100)	0.843104
	dpROF (x500)	0.843003
	dpROF (x1000)	0.842852
PSNR(image, ·)	dpROF	29.581741
	dpROF (x10)	29.581742
	dpROF (x50)	29.581747
	dpROF (x100)	29.581734
	dpROF (x500)	29.581242
	dpROF (x1000)	29.580261

Table 43: Different accuracy levels for the initial ROF calculation, $\sigma^2 = 0.004$, $\varepsilon = 10^{-6}$

Compared metric	Change in ROF accuracy	Value
Time	dpROF	2.62s + 9.61s
	dpROF (x10)	1.24s + 9.64s
	dpROF (x50)	0.76s + 9.65s
	dpROF (x100)	0.62s + 9.64s
	dpROF (x500)	0.62s + 9.63s
	dpROF (x1000)	0.38s + 9.64s
LPIPS(dpROF, ·)	dpROF (x10)	1.1×10^{-10}
	dpROF (x50)	1.5×10^{-9}
	dpROF (x100)	4.7×10^{-9}
	dpROF (x500)	6.8×10^{-8}
	dpROF (x1000)	2.2×10^{-7}
SSIM(dpROF, ·)	dpROF (x10)	$1 - 1.0 \times 10^{-10}$
	dpROF (x50)	$1 - 1.2 \times 10^{-9}$
	dpROF (x100)	$1 - 3.4 \times 10^{-9}$
	dpROF (x500)	$1 - 4.0 \times 10^{-8}$
	dpROF (x1000)	$1 - 1.2 \times 10^{-7}$
PSNR(dpROF, ·)	dpROF (x10)	127.89
	dpROF (x50)	116.90
	dpROF (x100)	112.23
	dpROF (x500)	100.98
	dpROF (x1000)	95.95
LPIPS(image, ·)	dpROF	0.17506009
	dpROF (x10)	0.17506018
	dpROF (x50)	0.17506054
	dpROF (x100)	0.17506107
	dpROF (x500)	0.17506574
	dpROF (x1000)	0.17507101
SSIM(image, ·)	dpROF	0.84311745
	dpROF (x10)	0.84311745
	dpROF (x50)	0.84311741
	dpROF (x100)	0.84311734
	dpROF (x500)	0.84311656
	dpROF (x1000)	0.84311571
PSNR(image, ·)	dpROF	29.5816072
	dpROF (x10)	29.5816073
	dpROF (x50)	29.5816076
	dpROF (x100)	29.5816079
	dpROF (x500)	29.5816083
	dpROF (x1000)	29.5816096

Table 44: Different accuracy levels for the initial ROF calculation, $\sigma^2 = 0.04$, $\varepsilon = 10^{-4}$

Compared metric	Change in ROF accuracy	Value
Time	dpROF	0.60s + 2.29s
	dpROF (x10)	0.31s + 2.31s
	dpROF (x50)	0.19s + 2.31s
	dpROF (x100)	0.15s + 2.31s
	dpROF (x500)	0.15s + 2.30s
	dpROF (x1000)	0.09s + 2.30s
LPIPS(dpROF, ·)	dpROF (x10)	2.7×10^{-6}
	dpROF (x50)	3.4×10^{-5}
	dpROF (x100)	9.5×10^{-5}
	dpROF (x500)	0.000813
	dpROF (x1000)	0.001956
SSIM(dpROF, ·)	dpROF (x10)	$1 - 1.5 \times 10^{-6}$
	dpROF (x50)	0.999980
	dpROF (x100)	0.999943
	dpROF (x500)	0.999478
	dpROF (x1000)	0.998710
PSNR(dpROF, ·)	dpROF (x10)	84.54
	dpROF (x50)	73.06
	dpROF (x100)	68.41
	dpROF (x500)	58.44
	dpROF (x1000)	54.37
LPIPS(image, ·)	dpROF	0.411805
	dpROF (x10)	0.411810
	dpROF (x50)	0.411839
	dpROF (x100)	0.411896
	dpROF (x500)	0.412218
	dpROF (x1000)	0.412613
SSIM(image, ·)	dpROF	0.629350
	dpROF (x10)	0.629336
	dpROF (x50)	0.629280
	dpROF (x100)	0.629209
	dpROF (x500)	0.628579
	dpROF (x1000)	0.627722
PSNR(image, ·)	dpROF	24.010069
	dpROF (x10)	24.009938
	dpROF (x50)	24.009402
	dpROF (x100)	24.008722
	dpROF (x500)	24.003141
	dpROF (x1000)	23.996046

Table 45: Different accuracy levels for the initial ROF calculation, $\sigma^2 = 0.04$, $\varepsilon = 10^{-6}$

Compared metric	Change in ROF accuracy	Value
Time	dpROF	1.24s + 5.05s
	dpROF (x10)	0.61s + 5.07s
	dpROF (x50)	0.38s + 5.06s
	dpROF (x100)	0.31s + 5.06s
	dpROF (x500)	0.31s + 5.08s
	dpROF (x1000)	0.20s + 5.08s
LPIPS(dpROF, ·)	dpROF (x10)	5.0×10^{-9}
	dpROF (x50)	5.5×10^{-8}
	dpROF (x100)	1.4×10^{-7}
	dpROF (x500)	1.3×10^{-6}
	dpROF (x1000)	3.6×10^{-6}
SSIM(dpROF, ·)	dpROF (x10)	1 - 3.0×10^{-9}
	dpROF (x50)	1 - 3.1×10^{-8}
	dpROF (x100)	1 - 8.1×10^{-8}
	dpROF (x500)	1 - 7.5×10^{-7}
	dpROF (x1000)	1 - 2.0×10^{-6}
PSNR(dpROF, ·)	dpROF (x10)	111.58
	dpROF (x50)	101.52
	dpROF (x100)	97.45
	dpROF (x500)	87.68
	dpROF (x1000)	83.26
LPIPS(image, ·)	dpROF	0.41180321
	dpROF (x10)	0.41180330
	dpROF (x50)	0.41180366
	dpROF (x100)	0.41180407
	dpROF (x500)	0.41180697
	dpROF (x1000)	0.41181000
SSIM(image, ·)	dpROF	0.62935017
	dpROF (x10)	0.62935004
	dpROF (x50)	0.62934929
	dpROF (x100)	0.62934836
	dpROF (x500)	0.62934172
	dpROF (x1000)	0.62933432
PSNR(image, ·)	dpROF	24.0100746
	dpROF (x10)	24.0100734
	dpROF (x50)	24.0100672
	dpROF (x100)	24.0100593
	dpROF (x500)	24.0099995
	dpROF (x1000)	24.0099283

References

- [1] K. Bredies. Recovering piecewise smooth multichannel images by minimization of convex functionals with total generalized variation penalty. In *Efficient Algorithms for Global Optimization Methods in Computer Vision*. Springer (2014) pp. 44–77.
- [2] Kristian Bredies, Karl Kunisch, Thomas Pock. Total generalized variation. *SIAM J. Imaging Sci.* 3 (2010) pp. 492–526. doi:10.1137/090769521.
- [3] Antoni Buades, Bartomeu Coll, Jean-Michel Morel. Non-Local Means Denoising. *Image Processing On Line* 1 (2011) pp. 208–212. https://doi.org/10.5201/ipol.2011.bcm_nlm.
- [4] Antonin Chambolle, Pierre-Louis Lions. Image recovery via total variation minimization and related problems. *Numer. Math.* 76 (1997) pp. 167–188. doi:10.1007/s002110050258.
- [5] Antonin Chambolle, Thomas Pock. A first-order primal-dual algorithm for convex problems with applications to imaging. *J. Math. Imaging Vis.* 40 (2011) pp. 120–145. doi:10.1007/s10851-010-0251-1.
- [6] Y. Chen, S. Levine, M. Rao. Variable exponent, linear growth functionals in image restoration. *Siam J. Appl. Math.* 66 (2006) pp. 1383–1406. doi:10.1137/050624522.
- [7] Leonid I. Rudin, Stanley Osher, Emad Fatemi. Nonlinear total variation based noise removal algorithms. *Phys. D* 60 (1992) pp. 259–268. doi:10.1016/0167-2789(92)90242-F. Experimental mathematics: computational issues in nonlinear science (Los Alamos, NM, 1991).
- [8] David Salomon. *Data Compression: The Complete Reference (4 ed.)*. Springer (2007).
- [9] Z. Wang, A.C. Bovik, H.R. Sheikh, E.P. Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing* 13 (2004) pp. 600–612.
- [10] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2018) pp. 584–593.